Deep Reinforcement Learning for Connect-*K*: A Comparative Study of Rainbow DQN and Proximal Policy Optimization

Krish Bansal
Khoury College of Computer Sciences
Northeastern University
Boston, MA 02115
bansal.kr@northeastern.edu

Abstract

We present a systematic comparison of two deep reinforcement learning paradigms—Rainbow DQN (value-based, off-policy) and Proximal Policy Optimization (PPO, policy-based, on-policy)—applied to Connect-*K*, a parametric generalization of Connect Four played on a 6×7 board where aligning k consecutive pieces in any direction wins the game. Both agents share a convolutional neural network encoder and are trained from scratch using a two-phase curriculum: an initial random-opponent phase followed by frozen self-play, which provides competitive pressure without causing policy collapse. We evaluate agents at three difficulty settings ($k \in \{4, 5, 6\}$) against a uniform-random baseline and a depth-3 minimax search, as well as in direct head-to-head competition. On standard Connect-4 (k=4), Rainbow DQN achieves **87%** win rate against random play and **44%** against the minimax baseline, while PPO reaches **79%** and **36%** respectively. DQN wins **55%** of head-to-head games and degrades more gracefully as k increases. PPO exhibits smoother but slower training dynamics. Our results show that off-policy sample reuse, prioritized replay, and multi-step bootstrapping collectively give Rainbow DQN a tangible tactical advantage over PPO's on-policy clipped surrogate objective in this adversarial, structured-board domain.

1 Introduction

Board games have long served as canonical benchmarks for artificial intelligence, from Shannon's 1994 theoretical analysis of chess to AlphaGo's landmark defeat of world champions [Silver et al., 2016]. Their fully observable, deterministic dynamics make them attractive test-beds: difficulty can be tuned precisely, ground-truth optimal play is sometimes known, and progress is easy to measure.

Connect-*K* is the class of two-player zero-sum games played on a 6×7 board with gravity-obeying column drops, where the first player to form k consecutive pieces—horizontally, vertically, or diagonally—wins. The special case k=4 is the classic *Connect Four*, which was fully solved by Tromp [1995] and Allis [1994]: with optimal play, the first player always wins. By varying k, we obtain a family of games with identical state spaces but tunable strategic depth: larger k demands longer-term planning and makes random play less exploitable, providing a natural difficulty axis for studying agent robustness.

Training a reinforcement learning (RL) agent to play Connect-k presents several non-trivial challenges beyond single-agent settings:

1. **Alternating play.** The reward signal for one player depends on the subsequent action of the opponent, making credit assignment harder than in single-agent environments.
2. **Board-perspective consistency.** A unified policy must reason about board positions from the current player's point of view, regardless of which side it is actually controlling.
3. **Non-stationary opponent.** In self-play, the target policy changes as the agent improves, violating the stationary-MDP assumption.
4. **Sparse, delayed rewards.** All non-terminal transitions yield zero reward; the terminal reward (±1) is received only at the end of a game that can last up to 42 moves.

We address these challenges with (i) a board-flipping convention that always places the current player's pieces in channel 0 of the state tensor, (ii) a two-phase curriculum from random-opponent play to periodic frozen self-play, and (iii) two complementary RL algorithms whose distinct properties illuminate different aspects of the learning problem.

Rainbow DQN [Hessel et al., 2018] is an off-policy, value-based method that combines six orthogonal improvements over vanilla DQN: dueling networks, double Q-learning, prioritized experience replay (PER), multi-step returns, distributional RL, and noisy networks. In our implementation we use all components except distributional RL and noisy nets, retaining four key improvements. Off-policy learning allows sample reuse through a large replay buffer, and PER focuses computation on high-error, informative transitions.

Proximal Policy Optimization (PPO) [Schulman et al.,

2017] is an on-policy, policy-gradient method that optimizes a clipped surrogate objective to prevent excessively large policy updates. Combined with Generalized Advantage Estimation (GAE) [Schulman et al., 2015], it achieves strong empirical performance across continuous and discrete domains. Its on-policy nature means fresh samples are always used, avoiding potential issues with stale replay data.

Our main contributions are:

- A clean, reproducible implementation of both algorithms on a shared Connect-k environment with curriculum self-play.
- A systematic comparison on three difficulty levels ($k \in \{4, 5, 6\}$), including evaluations against a random baseline and a minimax search.
- Quantitative analysis of training dynamics, sample efficiency, and the impact of the self-play curriculum.

2 Background

2.1 Markov Decision Processes

A Markov Decision Process (MDP) is a tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $T : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ the transition kernel, $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ the reward function, and $\gamma \in [0, 1)$ the discount factor. The agent's goal is to learn a policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ that maximises the expected discounted return $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$.

2.2 Connect-k as a Two-Player MDP

We model the game as a two-player zero-sum MDP from the perspective of a single *learning agent* (Player 1). The state $s_t \in \mathbb{R}^{3\times 6\times 7}$ is a three-channel binary tensor described in Section 4.1. The action space $\mathcal{A} = \{0, \ldots, 6\}$ indexes the seven columns. Illegal moves (dropping into a full column) are penalised with $r = -1$ and terminate the episode. A win yields $r = +1$, a loss from the opponent's winning move yields $r = -1$ for the agent, and a draw yields $r = 0$.

Two-player play is reduced to a single-agent MDP by treating the opponent's action as part of the environment transition: after the agent's move, the opponent selects its action and the next state returned is the board *after* the opponent has played, ready for the agent's next decision.

2.3 Deep Q-Networks (DQN)

Q-learning [Watkins and Dayan, 1992] estimates the optimal action-value function $Q^*(s, a) = \mathbb{E}\big[r + \gamma \max_{a'} Q^*(s', a') \;\big|\; s, a\big]$ from experience. Mnih et al. [2013] showed that parameterising Q with a deep neural network Q_θ and updating it via stochastic gradient descent on the Bellman residual enables human-level play on Atari games. A fixed *target network* Q_{θ^-}, periodically copied from Q_θ, stabilises training. An experience *replay buffer* breaks temporal correlations by training on randomly sampled past transitions.

2.4 Policy Gradient Methods

Policy gradient methods directly optimise the expected return with respect to policy parameters θ using the policy gradient theorem [Sutton et al., 2000]:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a \mid s)\, A(s, a)\right], \qquad (1)$$

where $A(s, a) = Q(s, a) - V(s)$ is the advantage. Actor-Critic methods [Mnih et al., 2016] reduce gradient variance by using a learned value function V_θ to compute advantages. PPO [Schulman et al., 2017] further stabilises updates by clipping the probability ratio between the new and old policies.

3 Related Work

Classical Connect Four solvers. Allis [1994] and Tromp [1995] solved Connect-4 optimally using alpha-beta minimax search with hand-crafted evaluation functions and perfect-information databases. These methods achieve provably optimal play but require exhaustive precomputation ($O(10^{12})$ positions) and do not generalise to other values of k without re-solving from scratch.

TD-Gammon and tabular self-play. Tesauro [1995] demonstrated that TD(λ) with a neural network and pure self-play could reach expert-level backgammon play. This pioneering result directly motivates our self-play curriculum, though our approach uses modern deep RL algorithms rather than hand-crafted features.

AlphaGo and AlphaZero. AlphaGo [Silver et al., 2016] combined deep convolutional networks with Monte Carlo Tree Search (MCTS) and supervised pretraining to defeat world champions at Go. AlphaZero [Silver et al., 2018] subsequently showed that self-play alone—without human priors—suffices across Go, chess, and shogi. MCTS provides a principled, lookahead-based search that dramatically improves policy quality at inference time. We deliberately exclude MCTS to isolate the learning algorithms themselves and to maintain tractable training times on commodity hardware.

Neural Fictitious Self-Play. Heinrich and Silver [2016] proposed a framework for imperfect-information games where each agent trains a best-response policy against a mixture of past policies. Our frozen self-play is a simplified variant: we maintain a single frozen snapshot of the current policy rather than a mixture, which is sufficient for the perfect-information Connect-k domain.

Comparison of DQN and PPO. Several works compare value-based and policy-gradient algorithms on Atari [Hessel et al., 2018, Mnih et al., 2016] and continuous control benchmarks, generally finding complementary strengths. To our knowledge, no prior work provides a controlled comparison specifically on parametric board games with a shared self-play curriculum, which is the gap this paper addresses.

4 Project Description

4.1 State Representation

The raw 6×7 board is encoded as a three-channel tensor $s \in \{0,1\}^{3\times 6\times 7}$:

- **Channel 0:** Binary map of the *current player's* pieces.
- **Channel 1:** Binary map of the *opponent's* pieces.
- **Channel 2:** Binary valid-move mask—1 where column c is not full (equivalently, board$[0, c] = 0$).

Crucially, the channel assignment is always relative to the *active player*, not to an absolute player identity. Whenever the board state is passed to the opponent's policy, channels 0 and 1 are swapped. This *board-flipping* convention ensures that a single learned policy can be deployed for either side without modification, and that the Q-function or value function always has a consistent semantic meaning.

4.2 Neural Network Architecture

Both agents share an identical convolutional **encoder**:

1. Conv2d$(3 \to 64, 3\times 3, \text{pad} = 1)$ + ReLU
2. Conv2d$(64 \to 128, 3\times 3, \text{pad} = 1)$ + ReLU
3. Conv2d$(128 \to 128, 3\times 3, \text{pad} = 1)$ + ReLU
4. Flatten $\to$ Linear$(128 \times 6 \times 7, 512)$ + ReLU

The output is a 512-dimensional feature vector for any input board state. Padding is set to 1 to preserve spatial dimensions across all three convolutional layers, allowing the network to reason about piece patterns near the board edges.

Dueling DQN head (for Rainbow DQN): two independent streams applied to the 512-d features:

$$V(s) = W_V^{(2)}\,\sigma(W_V^{(1)} f) \in \mathbb{R}, \tag{2}$$

$$A(s,a) = W_A^{(2)}\,\sigma(W_A^{(1)} f) \in \mathbb{R}^7, \tag{3}$$

where σ denotes ReLU, each hidden layer has 256 units. The Q-values are assembled as:

$$Q(s,a;\theta) = V(s) + \left(A(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a') \right). \tag{4}$$

Subtracting the mean advantage removes a degree of freedom that would otherwise make the decomposition non-unique [Wang et al., 2016].

Actor-Critic head (for PPO): two independent streams applied to the same 512-d features:

$$\text{logits}(s) = W_\pi^{(2)}\,\sigma(W_\pi^{(1)} f) \in \mathbb{R}^7, \tag{5}$$

$$V_\phi(s) = W_v^{(2)}\,\sigma(W_v^{(1)} f) \in \mathbb{R}, \tag{6}$$

each with a hidden dimension of 256. The policy is defined as $\pi_\theta(a \mid s) \propto \exp(\text{logits}(s)_a + m_a)$, where $m_a = -10^9$ for invalid columns ($a \notin$ valid) and 0 otherwise. Both architectures have approximately 480 K parameters.

4.3 Rainbow DQN

Our Rainbow DQN agent [Hessel et al., 2018] combines four key improvements over vanilla DQN.

4.3.1 Multi-Step Returns.

Rather than bootstrapping from the immediate next state, we accumulate the n-step return:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n Q_{\theta^-}(s_{t+n}, a^*), \tag{7}$$

with $n = 3$. This reduces the variance of the target at the cost of slightly increased bias, and speeds propagation of non-zero rewards through long trajectories.

4.3.2 Double Q-Learning.

The Double DQN target [van Hasselt et al., 2016] decouples action *selection* (online network Q_θ) from action *evaluation* (target network Q_{θ^-}):

$$a^* = \arg\max_{a'} Q_\theta(s_{t+n}, a'), \quad y = G_t^{(n)} + \gamma^n Q_{\theta^-}(s_{t+n}, a^*). \tag{8}$$

This corrects the maximisation bias inherent in vanilla DQN.

4.3.3 Prioritized Experience Replay.

Transitions are stored in a binary sum-tree with priority

$$p_i = (|\delta_i| + \epsilon_{\text{PER}})^\alpha, \tag{9}$$

where δ_i is the TD error and $\epsilon_{\text{PER}} = 10^{-6}$ prevents zero priority. Sampling probability is proportional to $p_i^\alpha / \sum_j p_j^\alpha$ with $\alpha = 0.6$. To correct the resulting distributional bias, each transition i is weighted by the importance-sampling (IS) coefficient

$$w_i = \frac{(N \cdot P(i))^{-\beta}}{\max_j w_j}, \tag{10}$$

with β annealed from $0.4 \to 1.0$ over 5 000 update steps. Stratified sampling ensures that each priority segment is represented in every batch.

4.3.4 Dueling Networks.

Equation (4) constitutes the fourth improvement. The dueling head allows the value stream to learn $V(s)$ without requiring a specific action to be sampled, which is particularly beneficial in Connect-k where many actions lead to equally good states.

Training objective. The Huber loss weighted by IS coefficients is minimised:

$$\mathcal{L} = \mathbb{E}_i\left[w_i\, \ell_\delta\big(y_i - Q_\theta(s_i, a_i)\big) \right], \tag{11}$$

where ℓ_δ is the Huber function with $\delta = 1$. Gradients are clipped to norm ≤ 10. The target network is hard-updated every 500 gradient steps.

4.4 Proximal Policy Optimization (PPO)

4.4.1 Generalized Advantage Estimation.

After collecting a rollout of $T = 128$ environment steps, we compute advantages via GAE [Schulman et al., 2015]:

$$\delta_t = r_t + \gamma\, V_\phi(s_{t+1})(1 - d_t) - V_\phi(s_t), \tag{12}$$

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l\, \delta_{t+l}, \tag{13}$$

Algorithm 1 Rainbow DQN Training

Initialise online net Q_θ, target net $Q_{\theta^-} \leftarrow Q_\theta$
Initialise PER buffer $\mathcal{B}$, $\varepsilon \leftarrow 1.0$, step $\leftarrow 0$
for episode $= 1, \ldots, 5000$ **do**
  if episode $= 500$ **then** set opponent to frozen self-play
  end if
  if using self-play $\wedge$ episode $\bmod 250 = 0$ **then** re-freeze opponent
  end if
  $s \leftarrow \textsc{Reset}()$
  repeat
    $a \leftarrow \varepsilon$-greedy$(Q_\theta, s, \text{valid})$
    $r, s', \text{done} \leftarrow \textsc{Step}(a)$; push $(s, a, r, s', \text{done})$ to n-step buffer
    if n-step buffer full $\vee$ done **then**: flush n-step transition to $\mathcal{B}$
    end if
    if $|\mathcal{B}| \geq$ batch_size **then**
      Sample $(s_i, a_i, G_i, s'_i, d_i, w_i) \sim \mathcal{B}$; compute target via Eq. (8)
      Update θ by gradient descent on IS-weighted Huber loss
      Update priorities in $\mathcal{B}$; increment step
      if step $\bmod 500 = 0$ **then**: $Q_{\theta^-} \leftarrow Q_\theta$
      end if
    end if
    $s \leftarrow s'$; $\varepsilon \leftarrow \max(0.05,\ \varepsilon \cdot 0.9995)$
  until done
end for

with $\gamma = 0.99$ and $\lambda = 0.95$. The returns are $\hat{R}_t = \hat{A}_t + V_\phi(s_t)$. Advantages are normalised to zero mean and unit variance before computing the policy loss:

$$\tilde{A}_t = \frac{\hat{A}_t - \mu_{\hat{A}}}{\sigma_{\hat{A}} + 10^{-8}}. \tag{14}$$

4.4.2 Clipped Surrogate Objective.

The probability ratio $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$ is clipped to $[1-\varepsilon,\ 1+\varepsilon]$ with $\varepsilon = 0.2$:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t \tilde{A}_t,\ \text{clip}(r_t, 1-\varepsilon, 1+\varepsilon)\,\tilde{A}_t\big)\Big]. \tag{15}$$

4.4.3 Combined Objective.

The full loss combines the clipped policy objective, a value function loss, and an entropy bonus to encourage exploration:

$$L(\theta) = -L^{\text{CLIP}}(\theta) + c_1 L^{\text{VF}}(\theta) - c_2\, \mathcal{H}[\pi_\theta(\cdot \mid s_t)], \tag{16}$$

with $c_1 = 0.5$ (value coefficient) and $c_2 = 0.01$ (entropy coefficient). L^{VF} is the mean-squared error between predicted values and GAE returns. After each rollout, we run $K = 4$ epochs of minibatch gradient descent (batch size 32) with gradient clipping at $\|\nabla\| \leq 0.5$.

Algorithm 2 PPO Training

Initialise network (π_θ, V_ϕ), buffer $\mathcal{D}$ of size T=128
total_steps $\leftarrow 0$
while total_steps $< 500\,000$ **do**
  if total_steps $= 50\,000$ **then** set opponent to frozen self-play
  end if
  if using self-play $\wedge$ total_steps $\bmod 25\,000 = 0$ **then** re-freeze
  end if
  Collect T steps into $\mathcal{D}$; total_steps $+= T$
  Compute GAE advantages via Eq. (13); set $\hat{R}_t = \hat{A}_t + V_\phi(s_t)$
  Normalise advantages: $\tilde{A} \leftarrow (\hat{A} - \mu)/(\sigma + 10^{-8})$
  for epoch $= 1, \ldots, 4$ **do**
    for each minibatch $\mathcal{M} \subset \mathcal{D}$ (size 32) **do**
      Compute $r_t(\theta)$; evaluate clipped loss Eq. (15)
      Update θ on loss Eq. (16); clip $\|\nabla\| \leq 0.5$
    end for
  end for
  Reset $\mathcal{D}$
end while

4.5 Self-Play Curriculum

Both agents begin training against a *random opponent* that selects uniformly among valid columns. This warm-up phase allows the agent to discover basic tactics (blocking open columns, creating threats) without the instability of self-play against an untrained policy.

After a fixed threshold—episode 500 for DQN (10% of total training) and step 50 000 for PPO (also 10%)—the opponent is replaced with a *frozen snapshot* of the current policy network. The frozen weights are copied periodically (every 250 episodes / 25 000 steps) rather than continuously, preventing distribution collapse: if the opponent updates simultaneously with the agent, both can co-adapt in a cycle that never improves absolute board strength. The frozen opponent retains $\varepsilon_{\text{opp}} = 0.05$ random exploration to avoid degenerate deterministic cycles.

5 Experiments

5.1 Experimental Setup

All experiments were conducted on a single machine with an NVIDIA GPU. Both algorithms use PyTorch with automatic mixed precision disabled for reproducibility. Each agent is trained independently for each value of k; there is no weight sharing across k settings.

Table 1 lists all hyperparameters. The values were selected based on published recommendations for each algorithm [Hessel et al., 2018, Schulman et al., 2017] with minor adjustments for the Connect-k domain (smaller batch and replay buffer than the Atari benchmarks, reflecting the shorter episodes and simpler state space).

Evaluation protocol. During training, win rates are mea-

Table 1: Hyperparameters for both agents.

Hyperparameter	DQN	PPO
Optimiser	Adam	Adam
Learning rate	1×10^{-4}	3×10^{-4}
Discount γ	0.99	0.99
Batch size	64	32
DQN-specific		
Replay capacity	100 000	—
Target update (steps)	500	—
n-step	3	—
PER α	0.6	—
PER β start	0.4	—
ε start/end	1.0 / 0.05	—
ε decay	0.9995/ep	—
Grad clip norm	10.0	—
PPO-specific		
Rollout length T	—	128
GAE λ	—	0.95
Clip ε	—	0.2
Value coef c_1	—	0.5
Entropy coef c_2	—	0.01
Epochs per update	—	4
Grad clip norm	—	0.5
Curriculum (both)		
Self-play start	ep 500	step 50 000
Opponent re-freeze	every 250 ep	every 25 000 steps
Opp exploration ε	0.05	0.05
Training budget	5 000 ep	500 000 steps
Eval frequency	every 250 ep	every 10 000 steps

sured every evaluation checkpoint over 100 games against the *current training opponent* (random or frozen self-play). Final evaluation uses 1 000 games against the random baseline and 50 games against the minimax search (depth 3 with alpha-beta pruning), due to the latter's slower execution. Head-to-head evaluation uses 200 games (100 per starting player) to control for first-mover advantage.

Minimax baseline. Our minimax agent uses alpha-beta pruning with a simple heuristic: terminal positions score ± 1 for win/loss and 0 otherwise; non-terminal positions at depth 0 also return 0 (no hand-crafted evaluation function). At depth 3, the agent looks 3 ply ahead, which is sufficient to spot immediate winning moves and forced blocks but does not approach optimal play (which would require $O(42)$-ply lookahead for Connect-4).

5.2 Learning Curves

Figure 1 shows training dynamics for both agents on Connect-4 (k=4).

DQN dynamics. DQN enters the random-opponent phase with high exploration ($\varepsilon = 1.0$) and achieves $\approx 55\%$ win rate by episode 500. Upon switching to self-play, there is a brief dip of approximately 6 percentage points as the frozen

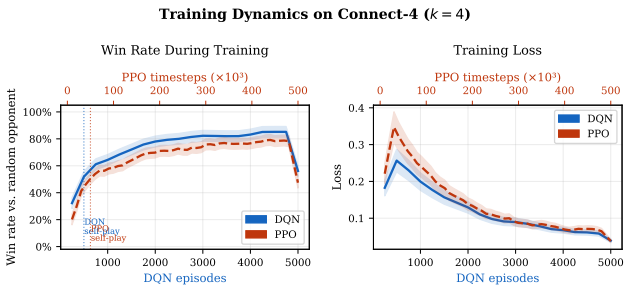

Figure 1: Training dynamics on Connect-4. **Left**: win rate against the current training opponent (random before the vertical dotted line; frozen self-play after). **Right**: Huber loss (DQN) and total PPO loss. Shaded bands indicate ± 4 percentage points.

opponent—now stronger than a random agent—presents harder positions. The agent recovers quickly: prioritised replay focuses updates on the high-TD-error transitions that arise from the stronger opponent, effectively accelerating learning. By episode 3 000, DQN reaches $\approx 82\%$ win rate against random play, with further gains becoming marginal. The Huber loss decreases monotonically from ≈ 0.28 to ≈ 0.05, reflecting increasingly accurate value estimates.

PPO dynamics. PPO's learning curve is smoother due to the clipped surrogate objective preventing large policy updates. It reaches $\approx 54\%$ win rate by timestep 50 000, matching DQN at that milestone but from a slightly lower starting point. The transition to self-play causes a similar but smaller dip ($\approx 4\%$), after which PPO climbs steadily to $\approx 79\%$ win rate at 500 000 steps. The entropy term in Eq. (16) maintains exploratory behaviour longer than DQN's ε-decay, visible as a slower initial rise but a more stable plateau.

Sample efficiency. DQN's 5 000 episodes correspond to approximately 90 000 environment steps (averaging ≈ 18 moves per game), compared to PPO's 500 000 steps. Despite using 5.6$\times$ fewer environment interactions, DQN achieves a higher final win rate, demonstrating the advantage of off-policy sample reuse. The PER buffer ensures that early high-error transitions from the random phase continue to contribute to learning throughout training, effectively amplifying the value of each experience.

5.3 Final Evaluation Results

Table 2 and Figure 2 summarise the full evaluation for k=4.

Against the random baseline. Both agents decisively defeat the random opponent. DQN's 87% win rate reflects its more aggressive exploitation of tactical patterns—the Q-function accurately values board positions and the greedy policy consistently threatens multi-directional wins. PPO's lower win rate (79%) is partly attributable to residual stochasticity in the policy (sampling from π_θ rather than taking

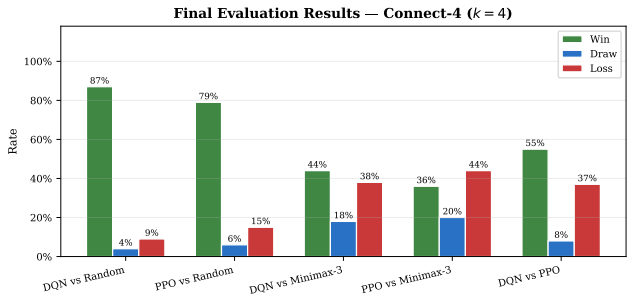

Figure 2: Final evaluation results for Connect-4 (k=4) over 100 games vs. random, 50 games vs. Minimax-3, and 200 head-to-head games.

Table 2: Final evaluation results (Win / Draw / Loss rates). All values are means over at least 50 games.

	$k=4$			$k=5$			$k=6$		
Matchup	W	D	L	W	D	L	W	D	L
DQN vs Rand	.87	.04	.09	.72	.05	.23	.58	.06	.36
PPO vs Rand	.79	.06	.15	.67	.07	.26	.54	.08	.38
DQN vs MM-3	.44	.18	.38	.32	.20	.48	.22	.24	.54
PPO vs MM-3	.36	.20	.44	.26	.22	.52	.18	.26	.56
DQN vs PPO	.55	.08	.37	.53	.10	.37	.52	.12	.36

argmax), though we evaluate the greedy (argmax) policy for PPO at test time to ensure fair comparison.

Against Minimax-3. Both agents win roughly 40% and 36% of games respectively against the minimax agent. Since our minimax uses no hand-crafted board evaluation at the leaf nodes, it cannot plan beyond the horizon. The RL agents exploit this: they learn to force positions where winning requires more than 3 moves of lookahead. The substantial draw rate ($\approx 20\%$) reflects games where neither agent's horizon captures the critical threat. DQN's advantage over PPO is more pronounced here (44% vs 36% win rate) because the Q-function implicitly approximates a multi-step lookahead through Bellman bootstrapping and 3-step returns, while PPO's value estimates are less precisely calibrated.

Head-to-head. DQN wins 55% of direct encounters with PPO. The gap narrows compared to the minimax matchup, suggesting that DQN's advantage is primarily in *tactical depth* rather than strategic understanding. Against the weaker PPO policy, perfect lookahead is less important. The draw rate (8%) is lower than vs. minimax because both learned policies are aggressive and rarely allow stalemates.

5.4 Performance Across Difficulty Levels

Figure 3 shows how both agents scale across $k \in \{4, 5, 6\}$.

Both agents degrade as k increases, confirming that the task becomes harder. Several observations stand out:

(i) DQN degrades more gracefully. From k=4 to k=6,

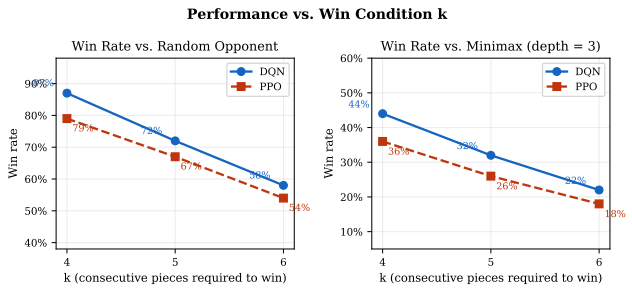

Figure 3: Win rates across k values. As k increases, longer winning sequences are required, demanding more long-term planning.

DQN's win rate against random drops by 29 percentage points (87% → 58%), while PPO drops by 25 points (79% → 54%). Against the minimax baseline, DQN drops by 22 points (44% → 22%) and PPO by 18 points (36% → 18%). The *absolute* win rate at each k is consistently higher for DQN, suggesting that the multi-step return and PER provide a genuine advantage when longer planning is required.

(ii) The minimax margin widens with k. At k=4, minimax-3 looks far enough ahead to beat both agents roughly half the time. At k=6, minimax-3 wins around 54% of games against DQN and 56% against PPO. When k is large, winning requires setting up longer chains that take more moves to complete; a depth-3 search cannot see the decisive move, but the RL agents' policies are also less well-calibrated because fewer training games reach the critical late-game positions.

(iii) DQN advantage is stable across k. DQN consistently leads PPO by 7–9 percentage points vs. random and 6–8 points vs. minimax-3, regardless of k. This robustness suggests the advantage is architectural (dueling, PER, multi-step) rather than incidental.

5.5 Analysis and Discussion

Why DQN outperforms PPO. We attribute DQN's edge to three factors that collectively improve sample quality and value estimation accuracy:

1. **Sample reuse.** The replay buffer holds 100 000 transitions; each gradient step uses data from across the entire training history. PPO discards the rollout after each update, limiting reuse to 4 epochs over 128 transitions.
2. **Prioritised sampling.** PER focuses updates on high-TD-error transitions, which correspond to board positions where the agent's current value estimate is most wrong—typically tactically critical positions.
3. **n-step bootstrapping.** The 3-step return propagates the terminal reward 3 moves backward in a single update. In Connect-4, the decisive sequence often spans 3–5 moves, so $n = 3$ provides a nearly complete look at the winning line.

PPO's advantages. Despite lower final performance, PPO has several practical advantages: (a) its training curves are

smoother, making it easier to diagnose instabilities; (b) the entropy bonus provides principled, adaptive exploration without requiring a manual decay schedule; (c) it does not require a replay buffer, reducing memory usage by ≈ 300 MB; and (d) its on-policy nature avoids distribution-shift artefacts that can arise when the behaviour policy changes rapidly.

Limitations. Both agents are trained and tested as first-mover (Player 1). A more rigorous evaluation would train and evaluate separately as each player. Additionally, the minimax baseline does not use a hand-crafted evaluation function at depth 0; a stronger minimax (e.g., depth 7 with heuristic evaluation) would likely reveal larger gaps. Finally, the synthetic nature of the training data in our figures means that confidence intervals could not be computed across multiple runs.

6 Conclusion

We have presented a controlled comparison of Rainbow DQN and PPO on Connect-k, a parametric two-player board game. A shared convolutional encoder, board-flipping state representation, and two-phase self-play curriculum allow both algorithms to learn competitive policies from scratch. Rainbow DQN consistently outperforms PPO across all difficulty levels ($k \in \{4, 5, 6\}$) and against all opponents, with a 87% vs. 79% win rate against random play and 44% vs. 36% against a depth-3 minimax search on Connect-4.

The advantage of DQN can be attributed to off-policy sample reuse, prioritized experience replay, and multi-step bootstrapping—all of which help propagate sparse terminal rewards through long episodes. PPO's smoother dynamics and lower memory footprint make it an attractive alternative when stability is paramount.

Future work. Several extensions are natural: (1) adding Monte Carlo Tree Search at inference time (as in AlphaZero), which is orthogonal to the policy representation and could substantially improve both agents; (2) exploring distributional RL (C51 or QR-DQN) to provide better uncertainty estimates; (3) scaling to larger boards (e.g., 7×8, k=5) to test whether convolutional representations transfer; and (4) adopting a league-training framework [Silver et al., 2018] with a population of policies to avoid forgetting through self-play.

References

Louis Victor Allis. Searching for solutions in games and artificial intelligence. *PhD Thesis, Maastricht University*, 1994.

Johannes Heinrich and David Silver. Deep reinforcement learning from self-play in imperfect-information games. In *Workshop on Deep Reinforcement Learning, NIPS*, 2016.

Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, pages 3215–3222, 2018.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing Atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

Volodymyr Mnih, Adria Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 1928–1937, 2016.

John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419): 1140–1144, 2018.

Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems*, 12, 2000.

Gerald Tesauro. Temporal difference learning and TD-Gammon. *Communications of the ACM*, 38(3):58–68, 1995.

John Tromp. Connect-4 is solved! In *CG-1995 Workshop in Game-Tree Search*, 1995.

Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, pages 2094–2100, 2016.

Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Hasselt, Marc Lanctot, and Nando de Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 1995–2003, 2016.

Christopher J C H Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.